

Зашифрований текст:

-луулшщнлууюфчсб-лмгйїубтбзщншкслчфіммпфчйшуфшщлєркгрбшуйпб

Таблиця розшифрування та перевірка

Таблиця розшифрування є оберненою до таблиці шифрування: у верхньому рядку — шифрований алфавіт, у нижньому — відкритий.

б	в	г	г	д	є	ж	з	и	ї	й	л	м	о	п	р	с	у	ф	ц	ч	ш	щ	ь	ю	я	-	т	е	х	н	і	к	а
а	б	в	г	г	д	е	є	ж	з	и	і	ї	й	к	л	м	н	о	п	р	с	т	у	ф	х	ц	ч	ш	щ	ь	ю	я	-

Розшифрований текст:

цінність інформації визначається мірою її корисності для власника

Розшифрований текст збігається з вихідним відкритим повідомленням, що підтверджує правильність виконання шифрування та дешифрування.

		Рижченко Я.В.			ДУ «Житомирська політехніка». 26.121.20.000 – ПрЗ	Арк.
		Лобанчикова Н.М.				
Змн.	Арк.	№ докум.	Підпис	Дата		2

Завдання 3. Шифр вертикальної перестановки

Ключове слово: «Астероїд». Текст: «Двері та вікна приміщення серверної кімнати повинні бути обов'язково екранованими».

Створення ключа

Нумерація стовпців: кожній літері ключового слова присвоюється порядковий номер відповідно до алфавітного порядку літери (менша за алфавітом літера отримує менший номер).

а	с	т	е	р	о	ї	д
1	6	7	3	5	4	8	2

Шифрування

Канонічний вигляд тексту (71 символ):

дверітавікнаприміщеннясерверноїкімнатиповиннібутиобов'язковоекранованими

Розмір таблиці: 9 рядків × 8 стовпців.

Вписуємо текст у таблицю послідовно рядок за рядком:

1	6	7	3	5	4	8	2
а	с	т	е	р	о	ї	д
д	в	е	р	і	т	а	в
і	к	н	а	п	р	и	м
і	щ	е	н	н	я	с	е
р	в	е	р	н	о	ї	к
і	м	н	а	т	и	п	о
в	и	н	н	і	б	у	т

и	о	б	о	в	я	з	к
о	в	о	е	к	р	а	н
о	в	а	н	и	м	и	

Випишуємо літери зі стовпців у порядку їх номерів (1, 2, 3, ...):

Стовпець №1 (а): діірівіоо

Стовпець №2 (д): вмекоткн

Стовпець №3 (е): ранраноен

Стовпець №4 (о): тряоибярм

Стовпець №5 (р): іпннтівки

Стовпець №6 (с): вкщвмиовв

Стовпець №7 (т): енееннбоа

Стовпець №8 (ї): аисїпузаи

Шифротекст:

діірі виоов мекот кнран раное нтряо ибярм іпннт івків кщвми оввен ееннб оааис
їпуза и

Дешифрування

Для розшифрування у порядку номерів стовпців вписуємо шифротекст у таблицю зверху вниз, потім випишуємо текст послідовно рядками.

Розшифрований текст:

дверітавікнаприміщеннясерверноїкімнатиповиннібутиобовязковоекранованим
и

Розшифрований текст збігається з вихідним відкритим повідомленням.

		Рижченко Я.В.			ДУ «Житомирська політехніка».26.121.20.000 – Лр3	Арк.
		Лобанчикова Н.М.				
Змн.	Арк.	№ докум.	Підпис	Дата		4

Завдання 4. Шифр гамування

Формула шифрування: $S = (O - \Gamma) \bmod 26$.

Формула розшифрування: $O = (S + \Gamma) \bmod 26$.

Відкритий текст: «If access to information is restricted then such information is confidential».

Слово-гасло (прізвище студента): rizhenko.

Таблиця кодування символів

Кожній літері латинського алфавіту присвоюється числовий код від 0 до 25:

a	b	c	d	e	f	g	h	i	j	k	l	m
0	1	2	3	4	5	6	7	8	9	10	11	12

n	o	p	q	r	s	t	u	v	w	x	y	z
13	14	15	16	17	18	19	20	21	22	23	24	25

Шифрування

Канонічний вигляд відкритого тексту (тільки латинські літери, нижній регістр):

ifacesstoinformationisrestrictedthensuchinformationisconfidential

Довжина: 66 символів.

Гама (циклічне повторення гасла «rizhenko» до довжини тексту):

rizhenkorizhenkorizhenkorizhenkorizhenkorizhenkorizhenkorizhenkori

Таблиця шифрування (перші 15 символів наведено у покроковому вигляді):

№	О	Г	Код О	Код Г	О – Г	mod 26	S
1	i	r	8	17	-9	17	r
2	f	i	5	8	-3	23	x
3	a	z	0	25	-25	1	b
4	c	h	2	7	-5	21	v
5	c	e	2	4	-2	24	y
6	e	n	4	13	-9	17	r
7	s	k	18	10	8	8	i
8	s	o	18	14	4	4	e

9	t	r	19	17	2	2	c
10	o	i	14	8	6	6	g
11	i	z	8	25	-17	9	j
12	n	h	13	7	6	6	g
13	f	e	5	4	1	1	b
14	o	n	14	13	1	1	b
15	r	k	17	10	7	7	h

Повна таблиця шифрування:

O	Ifacesstoinformationisrestric
Г	rizhenkorizhenkorizhenkorizhen
S	rxbvyriecggbbhyjljhjvidnkukep

O	tedthensuchinformationisconfid
Г	korizhenkorizhenkorizhenkorizh
S	jqmlxjfkqoaykecmcapgefsawxjw

O	ential
Г	enkori
S	aajujd

Шифротекст:

rxbvyriecggbbhyjljhjvidnkukepjqmlxjfkqoaykecmcapgefsawxjwaajujd

		Рижченко Я.В.			ДУ «Житомирська політехніка».26.121.20.000 – ЛрЗ	Арк.
		Лобанчикова Н.М.				6
Змн.	Арк.	№ докум.	Підпис	Дата		

Розшифрування

Розшифрування виконується за формулою: $O = (S + \Gamma) \bmod 26$.

S	rxbvryiecggbbhyjljhjvidnkukep
Г	rizhenkorizhenkorizhenkorizhen
О	ifacesstoinformationisrestric

S	jqmlxjfkqaooykecmcapgefsawxjw
Г	korizhenkorizhenkorizhenkorizh
О	tedthensuchinformationisconfid

S	aajujd
Г	enkori
О	ential

Розшифрований текст:

ifacesstoinformationisrestrictedthensuchinformationisconfidential

Розшифрований текст збігається з канонічним виглядом відкритого повідомлення, що підтверджує правильність виконання шифрування та дешифрування.

		Рижченко Я.В.			ДУ «Житомирська політехніка».26.121.20.000 – ЛрЗ	Арк.
		Лобанчикова Н.М.				
Змн.	Арк.	№ докум.	Підпис	Дата		7

Завдання 5. Оцінювання стійкості паролів

Оцінюємо стійкість паролів двох зареєстрованих користувачів за алгоритмом, наведеним у теоретичній частині.

Пароль 1: «qwerty123»

Крок	Умова	Результат	U
1	Довжина $L = 9$ ($8 \leq L \leq 15$)	$U = 12$	12
2	Тільки нижній регістр	$U + 5$	17
3	Кількість цифр $N = 3$ ($N \geq 3$)	$U + 7$	24
4	Спецсимволів немає	+0	24
5	Відсутні ≥ 2 категорії	+0	24
6	$U = 24$ ($15 < U < 25$)	СЛАБКИЙ	24

Пароль 2: «MyP@ssw0rd##2024»

Крок	Умова	Результат	U
1	Довжина $L = 16$ ($L \geq 16$)	$U = 18$	18
2	Обидва регістри (є малі і великі)	$U + 7$	25
3	Кількість цифр $N = 5$ ($N \geq 3$)	$U + 7$	32
4	Спецсимволів $S = 3$ ($S \geq 2$)	$U + 10$	42
5	Всі категорії присутні	$U + 6$	48
6	$U = 48$ ($U > 44$)	ДУЖЕ СИЛЬНИЙ	48

Висновок щодо зберігання паролів у вигляді хеш-функцій

		Риженко Я.В.			ДУ «Житомирська політехніка».26.121.20.000 – ЛрЗ	Арк.
		Лобанчикова Н.М.				8
Змн.	Арк.	№ докум.	Підпис	Дата		

У великих інформаційних системах паролі зберігаються у вигляді значень хеш-функцій (наприклад, bcrypt, SHA-256 із сіллю) з таких причин:

1) Хешування є одnobічною операцією — відновити пароль з хеш-значення математично неможливо (на відміну від шифрування, де знаючи ключ можна розшифрувати).

2) Навіть якщо база даних скомпрометована, зловмисник отримає лише хеші, а не реальні паролі.

3) Застосування «солі» (унікального випадкового рядка, що додається до пароля перед хешуванням) унеможлиблює використання Rainbow Table атак.

4) При автентифікації система хешує введений пароль і порівнює результат зі збереженим хешем — паролі у відкритому вигляді ніколи не зберігаються і не передаються.

		Рижченко Я.В.			ДУ «Житомирська політехніка».26.121.20.000 – ЛрЗ	Арк.
		Лобанчикова Н.М.				
Змн.	Арк.	№ докум.	Підпис	Дата		9

Завдання 6. Програмна реалізація шифру гамування

Програмна реалізація виконана мовою C# і реалізує повний цикл: канонізацію тексту, побудову гами, шифрування за формулою $S = (O - \Gamma) \bmod 26$ та розшифрування за формулою $O = (S + \Gamma) \bmod 26$. Текст і гасло вводяться програмно — або через аргументи командного рядка, або інтерактивно з клавіатури.

Програма складається з чотирьох методів:

- Canonicalize(string text) — видаляє всі символи, що не є латинськими літерами, та приводить рядок до нижнього регістру.
- BuildGamma(string key, int length) — циклічно повторює гасло до потрібної довжини.
- Encrypt(string plaintext, string gamma) — шифрування за формулою $S = (O - \Gamma) \bmod 26$.
- Decrypt(string ciphertext, string gamma) — розшифрування за формулою $O = (S + \Gamma) \bmod 26$.

Лістинг програми:

```
using System;
using System.Text;

namespace GammaCipher
{
    class Program
    {
        private const string Alphabet = "abcdefghijklmnopqrstuvwxyz";

        static string Canonicalize(string text)
        {
            var sb = new StringBuilder();
            foreach (char c in text)
            {
                if (char.IsLetter(c) && c < 128)
                    sb.Append(char.ToLower(c));
            }
            return sb.ToString();
        }

        static string BuildGamma(string key, int length)
        {
            var sb = new StringBuilder();
            for (int i = 0; i < length; i++)
                sb.Append(key[i % key.Length]);
            return sb.ToString();
        }

        static string Encrypt(string plaintext, string gamma)
```

		Рижченко Я.В.			ДУ «Житомирська політехніка».26.121.20.000 – Лр3	Арк.
		Лобанчикова Н.М.				10
Змн.	Арк.	№ докум.	Підпис	Дата		

```

{
    var sb = new StringBuilder();
    for (int i = 0; i < plaintext.Length; i++)
    {
        int o = Alphabet.IndexOf(plaintext[i]);
        int g = Alphabet.IndexOf(gamma[i]);
        int s = ((o - g) % 26 + 26) % 26;
        sb.Append(Alphabet[s]);
    }
    return sb.ToString();
}

static string Decrypt(string ciphertext, string gamma)
{
    var sb = new StringBuilder();
    for (int i = 0; i < ciphertext.Length; i++)
    {
        int s = Alphabet.IndexOf(ciphertext[i]);
        int g = Alphabet.IndexOf(gamma[i]);
        int o = (s + g) % 26;
        sb.Append(Alphabet[o]);
    }
    return sb.ToString();
}

static void PrintTable(string source, string gamma, string result, string formula, int maxRows = 15)
{
    Console.WriteLine($"{",-4} {"Char",-6} {"Key",-6} {"Code C",-8} {"Code K",-8} {"Expr",-10} {"mod26",-7} {"Result",-6}");
    Console.WriteLine(new string('-', 60));
    int count = Math.Min(maxRows, source.Length);
    for (int i = 0; i < count; i++)
    {
        int src = Alphabet.IndexOf(source[i]);
        int g = Alphabet.IndexOf(gamma[i]);
        int expr = formula == "encrypt" ? src - g : src + g;
        int mod = ((expr % 26) + 26) % 26;
        Console.WriteLine($"{i + 1,-4} {source[i],-6} {gamma[i],-6} {src,-8} {g,-8} {expr,-10} {mod,-7} {result[i],-6}");
    }
    if (source.Length > maxRows)
        Console.WriteLine($"... ({source.Length - maxRows} more characters omitted)");
}

static void Main(string[] args)
{
    Console.OutputEncoding = Encoding.UTF8;
    Console.WriteLine("=== Gamma Cipher ===\n");

    string inputText;
    string key;

    if (args.Length >= 2)
    {
        inputText = args[0];
        key = args[1].ToLower();
        Console.WriteLine($"Text : {inputText}");
        Console.WriteLine($"Key : {key}");
    }
}

```

		Риженко Я.В			ДУ «Житомирська політехніка».26.121.20.000 – Пр3	Арк.
		Лобанчикова Н.М.				11
Змн.	Арк.	№ докум.	Підпис	Дата		

```

    }
else
{
    Console.Write("Enter plaintext : ");
    inputText = Console.ReadLine() ?? "";
    Console.Write("Enter key      : ");
    key = (Console.ReadLine() ?? "").ToLower();
}

string canonicalKey = Canonicalize(key);
if (canonicalKey.Length == 0)
{
    Console.WriteLine("Error: key contains no Latin letters.");
    return;
}

string plaintext = Canonicalize(inputText);
Console.WriteLine($"[1] Canonical plaintext ({plaintext.Length} chars:");
Console.WriteLine($"    {plaintext}");

string gamma = BuildGamma(canonicalKey, plaintext.Length);
Console.WriteLine($"[2] Gamma (key \"{canonicalKey}\" repeated to {plaintext.Length} chars:");
Console.WriteLine($"    {gamma}");

string ciphertext = Encrypt(plaintext, gamma);
Console.WriteLine($"[3] Encryption  S = (O - K) mod 26 (first 15 steps:");
PrintTable(plaintext, gamma, ciphertext, "encrypt");
Console.WriteLine($"[4] Ciphertext:");
Console.WriteLine($"    {ciphertext}");

string decrypted = Decrypt(ciphertext, gamma);
Console.WriteLine($"[5] Decryption  O = (S + K) mod 26 (first 15 steps:");
PrintTable(ciphertext, gamma, decrypted, "decrypt");
Console.WriteLine($"[6] Decrypted text:");
Console.WriteLine($"    {decrypted}");

bool ok = decrypted == plaintext;
Console.WriteLine($"[7] Verification: decrypted text {(ok ? "matches plaintext" : "does NOT match")}");
}
}
}

```

		Рижченко Я.В.			ДУ «Житомирська політехніка».26.121.20.000 – ЛрЗ	Арк.
		Лобанчикова Н.М.				12
Змн.	Арк.	№ докум.	Підпис	Дата		

Результат виконання:

```
Microsoft Visual Studio Debu x + v
=== Gamma Cipher ===

Enter plaintext : If access to information is restricted then such information is confidential
Enter key       : rizhenko

[1] Canonical plaintext (66 chars):
    ifaccessstoinformationisrestrictedthensuchinformationisconfidential

[2] Gamma (Key "rizhenko" repeated to 66 chars):
    rizhenkorizhenkorizhenkorizhenkorizhenkorizhenkorizhenkorizhenkori

[3] Encryption S = (O - K) mod 26 (first 15 steps):
```

#	Char	Key	Code C	Code K	Expr	mod26	Result
1	i	r	8	17	-9	17	r
2	f	i	5	8	-3	23	x
3	a	z	0	25	-25	1	b
4	c	h	2	7	-5	21	v
5	e	n	4	13	-9	24	y
6	e	k	18	10	8	17	r
7	s	o	18	14	4	4	e
8	s	r	19	17	2	2	c
9	t	i	14	8	6	6	g
10	o	z	8	25	-17	9	j
11	i	h	13	7	6	6	j
12	n	e	5	4	1	1	b
13	f	n	14	13	1	1	b
14	o	k	17	10	7	7	h
15	r						

```
... (51 more characters omitted)
```

Ciphertext:
rxbyvriecggbbhyjljhjvidnkukepqmlixjfkoqaoykecmcapgefawxjwaaajujd

```
[4] Decryption O = (S + K) mod 26 (first 15 steps):
```

#	Char	Key	Code C	Code K	Expr	mod26	Result
1	r	r	17	17	34	8	i
2	x	i	23	8	31	5	f
3	b	z	1	25	26	0	a
4	v	h	21	7	28	2	c
5	y	e	24	4	28	2	c
6	r	n	17	13	30	4	e

```
... (51 more characters omitted)
```

6	r	n	17	13	30	4	e
7	i	k	8	10	18	18	s
8	e	o	4	14	18	18	s
9	c	r	2	17	19	19	t
10	g	i	6	8	14	14	o
11	j	z	9	25	34	8	i
12	g	h	6	7	13	13	n
13	b	e	1	4	5	5	f
14	b	n	1	13	14	14	o
15	h	k	7	10	17	17	r

```
... (51 more characters omitted)
```

Decrypted text:
ifaccessstoinformationisrestrictedthensuchinformationisconfidential

```
[5] Verification: decrypted text matches plaintext
```

Рис. 1-2. Результат виконання.

Висновок: У ході виконання лабораторної роботи були досліджені основні симетричні криптографічні алгоритми: шифр заміни, шифр вертикальної перестановки та шифр гамування. Для кожного алгоритму було виконано шифрування та дешифрування тексту, після чого розшифровані повідомлення у всіх випадках збіглися з вихідними, що підтвердило коректність розрахунків. Також була проведена оцінка стійкості паролів, яка показала, що використання різних категорій символів та достатня довжина суттєво підвищують рівень захисту. Додатково розглянуто принцип зберігання паролів у вигляді хеш-значень із застосуванням солі, що унеможлиблює відновлення паролів навіть у разі компрометації бази даних. На основі теоретичного матеріалу реалізовано програму мовою C#, яка автоматизує процес гамування — від канонізації тексту до перевірки результату дешифрування.